체인에서 그래프로

1-1. 체인의 한계
1-2. 그래프 설계 원리
1-3. State 스키마와 타입 힌트

1-1. 체인의 한계

> 한 방향으로 흐르는 체인으로는 못 만드는 흐름이 무엇인지 이해해봅니다.

1일차와 2일차에서 쓴 체인(LCEL)은 이런 모양이었습니다.

```
chain = prompt | llm | parser
result = chain.invoke({"topic": "재택근무 규정"})
```

데이터가 왼쪽에서 오른쪽으로 한 번 흐르고 끝납니다. 단방향 1회성 구조입니다. 번역, 요약, 분류처럼 입력을 넣으면 출력이 나오는 작업에는 이것으로 충분합니다.

하지만 실제 Agent가 풀어야 하는 일은 다르게 생겼습니다. "지난 분기 매출 보고서를 분석해 줘"라는 요청 하나에도 여러 갈래의 흐름이 섞여 있습니다.

- **데이터 조회**: 먼저 매출 데이터를 가져옵니다.
- **다시 조회(반복)**: 이상치가 있으면 앞 단계로 돌아가 다시 가져옵니다.
- **요약(분기)**: 보고서가 길면 요약하고 짧으면 그대로 둡니다.
- **되돌아가기(반복과 사람 개입)**: 사용자 검토 후 수정 요청이 오면 앞 단계로 되돌아갑니다.

체인으로는 이 흐름을 자연스럽게 표현할 수 없습니다. 구체적으로 네 가지가 안 됩니다.

1. 분기가 안 됩니다. 체인은 파이프라인이라 중간에 조건에 따라 경로가 갈리는 것을 표현하기 어렵습니다. 억지로 넣으려면 별도 도구가 필요하고 코드가 금방 복잡해집니다.

2. 반복이 안 됩니다. 체인은 순환이 없는 구조라 자기 자신으로 돌아갈 수 없습니다. 재시도를 하려면 체인 바깥에서 직접 루프를 돌려야 합니다.

```
# 체인 바깥에서 수동으로 루프를 돌리는 어색한 코드
result = chain.invoke(...)
while result["quality"] < 0.8:
    result = chain.invoke(...)
```

3. 상태가 없습니다. 이전 단계 결과를 다음 단계에서 쓰려면 매번 손으로 넘겨야 합니다.

```
step1 = chain1.invoke(...)
step2 = chain2.invoke({**inp, "from_step1": step1})
step3 = chain3.invoke({**inp, "from_step1": step1, "from_step2": step2})
# 단계가 늘수록 입력 dict가 계속 커집니다
```

4. 사람의 개입이 어렵습니다. 체인은 한번 시작하면 끝까지 흘러갑니다. 중간에 멈추고 승인을 기다렸다가 다시 시작하는 흐름이 자연스럽지 않습니다. 이 주제는 5일차의 HITL(Human-in-the-Loop, 사람 개입 승인)에서 다시 다룹니다.

자주 나오는 질문이 있습니다. "체인도 함수 안에서 if문을 쓰면 분기되는 것 아닌가요?" → 되기는 합니다.

문제는 그렇게 하는 순간 흐름 제어 로직이 체인 바깥의 일반 파이썬 코드에 흩어진다는 점입니다. 그러면 전체 흐름을 한눈에 볼 수 없고 그림으로 그릴 수도 없습니다. 중간에 멈췄다 다시 시작하는 것도 안 됩니다. 그래서 LangGraph는 이 흐름 제어 자체를 프레임워크 안으로 가져옵니다.

> 체인은 단방향 1회성이라 분기, 반복, 상태 공유, 사람 개입이 어렵다는 걸 확인했습니다.

1-2. 그래프 설계 원리

> 노드(작업), 엣지(흐름), 상태(공유 데이터)로 Agent의 동작을 그래프로 표현하는 원리를 이해해봅니다.

Agent의 동작을 그래프로 표현하면 분기, 반복, 상태 공유가 모두 자연스럽게 들어갑니다. 다섯 개의 단어만 기억하면 됩니다.

개념	한 줄 정의
State	모든 노드가 공유하는 데이터 통
Node	State를 읽고 갱신할 부분만 반환하는 함수 (작업 단위)
Edge	노드에서 노드로 가는 직진 화살표
Conditional Edge	State를 보고 다음 노드를 동적으로 결정하는 분기
Cycle	이전 노드로 돌아갈 수 있는 능력 (체인에는 없는 것)

State는 회의실 화이트보드에 비유하면 쉽습니다. 모든 노드(참석자)가 같은 보드를 보고 자기 차례가 오면 보드를 읽고 필요한 부분만 적거나 고친 뒤 다음 사람에게 넘깁니다. 데이터를 매번 손으로 전달할 필요가 없습니다.

체인과 그래프를 나란히 비교하면 이렇습니다.

측면	체인 (LCEL)	그래프 (LangGraph)
모델	파이프 (단방향)	그래프 (분기와 순환 가능)
분기	어색함	조건부 엣지로 자연스러움
반복	불가	같은 노드로 돌아가면 됨
상태	매번 수동 전달	State 객체로 자동 공유
사람 개입	어려움	interrupt 지원 (5일차)

LCEL과 LangGraph는 경쟁 관계가 아니라 역할이 다릅니다. LCEL은 노드 안에서 쓰는 단일 호출 파이프(직선)이고 LangGraph는 그 노드들을 잇는 지도입니다. 실제로 LangGraph 노드 함수 안에서 LCEL 체인을 그대로 호출할 수 있습니다.

```
chain = prompt | llm | parser          # LCEL: 직선

def llm_node(state: State) -> dict:    # LangGraph 노드 안에서 LCEL 사용
    result = chain.invoke({"input": state["query"]})
    return {"answer": result}
```

오늘 하루의 목표를 한 문장으로 줄이면 이렇습니다. LCEL은 한 번 흐르고 끝나지만 LangGraph는 돌고 분기한다는 것을 코드로 익힙니다.

> 노드, 엣지, 상태 세 부품으로 분기와 반복과 상태 공유를 모두 표현할 수 있다는 걸 확인했습니다.

1-3. State 스키마와 타입 힌트

> TypedDict로 상태 스키마를 정의하는 이유를 Java의 DTO와 비교해 이해해봅니다.

State는 파이썬의 `TypedDict` 로 정의합니다. 어떤 키가 있고 그 값이 어떤 타입인지 미리 적어 두는 딕셔너리라는 뜻입니다.

```
from typing import TypedDict

class State(TypedDict):
    query: str
    documents: list
    answer: str
```

`TypedDict` 는 Java의 DTO 클래스에 대응하는 개념입니다. 생김새가 비슷합니다.

```
// Java의 DTO
public class StateDto {
    private String query;
    private List<Document> documents;
    private String answer;
    // getter, setter ...
}
```

차이점도 알아 두면 혼란이 줄어듭니다.

- Java DTO는 진짜 클래스이고 인스턴스가 그 타입을 가집니다. 반면 `TypedDict` 는 실행 시점에는 그냥 dict입니다. `State(query="...")` 처럼 만들어도 실제로는 `{"query": "..."}` 딕셔너리일 뿐입니다.
- 타입 검사는 IDE와 정적 분석 도구(mypy 등)가 코드 작성 시점에 해 줍니다. 실행 중에 타입이 틀려도 파이썬은 에러를 내지 않습니다. 컴파일 타임 검사만 있고 런타임 검사는 없는 DTO라고 생각하면 됩니다.
- 런타임 검증까지 원하면 Pydantic BaseModel을 쓸 수도 있습니다. LangGraph는 둘 다 지원합니다. 이 과정에서는 표준 라이브러리만으로 되는 `TypedDict` 를 기본으로 씁니다.

타입을 선언하는 이유는 세 가지입니다.

1. 그래프의 계약서 역할을 합니다. 어떤 노드든 State에 어떤 키가 있는지 스키마만 보면 압니다. 노드가 10개가 되어도 데이터 구조가 한 곳에 정리되어 있습니다.

2. IDE 자동 완성과 오타 방지에 쓰입니다. `state["qeury"]` 같은 오타를 IDE가 잡아 줍니다.
3. LangGraph가 이 스키마를 실제로 읽습니다. 상태의 키 목록과 각 키의 병합 방법(reducer는 3번째 세션 3-1에서 다룹니다)을 이 클래스 정의에서 파악합니다. 단순한 문서가 아니라 프레임워크가 사용하는 설정입니다.

Java 감각으로 쓸 때 주의할 점이 두 가지 있습니다. 먼저 `state.query` 처럼 점 표기로 접근하면 `AttributeError` 가 납니다. 실행 시점에는 dict이므로 반드시 `state["query"]` 로 읽습니다.
또 모든 키를 처음부터 채울 필요는 없습니다. 다만 아직 값이 없는 키를 읽으면 `KeyError` 가 나므로 `state.get("attempts", 0)` 처럼 기본값을 주는 편이 안전합니다.

> State는 그래프의 계약서이고 실행 시점에는 평범한 dict라는 걸 확인했습니다.
